



Universidad de Oviedo
Grado en Física

Introducción a la Física Computacional

Curso 2023–24

Primeros pasos en GNU/Linux (Ubuntu)

Índice general

1. Primeros pasos en GNU/Linux (Ubuntu)	3
1.1. Objetivo	3
1.2. El sistema operativo Linux	3
1.2.1. Distribuciones de Linux	3
1.2.2. El entorno de trabajo Shell	4
1.2.3. El entorno de trabajo X Window	4
1.3. Entrar en sesión	5
1.4. Linux en modo gráfico	5
1.5. Linux en modo terminal	6
1.6. Trabajo básico con ficheros en modo terminal	9
1.7. Otros comandos interesantes	11
1.8. Ficheros estándar y redireccionamiento	12
1.8.1. Redirección simple	13
1.8.2. Redirección encadenada	14
1.9. Referencias absolutas y relativas en el árbol de directorios	14
1.10. Caracteres comodín en los nombres de ficheros	16
1.11. Propietarios de los ficheros y sus permisos de acceso	16
1.12. Ejecución de procesos en Unix	17
1.12.1. Lanzado de programas en modo normal	17
1.12.2. Lanzado de programas en modo desatendido	18

1.12.3. Control de ejecución de los procesos	19
--	----

Práctica 1

Primeros pasos en GNU/Linux (Ubuntu)

1.1. Objetivo

Proporcionar una visión general del sistema operativo *Linux*, en especial del entorno de trabajo en modo terminal presentando los comandos básicos necesarios que esta aplicación permite ejecutar.

1.2. El sistema operativo Linux

Linux es un sistema operativo gratuito y de libre distribución inspirado en el sistema *Unix*. Es un sistema operativo pensado para ser flexible, estable y fácilmente portable a diferentes tipos de ordenadores, por lo que existen versiones para *PC*, *Mac* y estaciones de trabajo.

1.2.1. Distribuciones de Linux

Con *Linux* aparece el concepto de *distribución*, que consiste en un agrupamiento software basado en el *núcleo Linux* (desarrollado por Linus Torvalds) que incluye determinados paquetes de software para satisfacer las necesidades de un grupo específico de usuarios, dando así origen a ediciones domésticas, empresariales y para servidores. Por lo general estas distribuciones están compuestas, total o mayoritariamente, de software libre, aunque a menudo incorporan aplicaciones o controladores propietarios.

Además del núcleo Linux, las distribuciones incluyen habitualmente las bibliotecas y herramientas del proyecto *GNU* y el sistema de ventanas *X Window System* (no *XWindows*). Dependiendo del tipo de usuarios a los que la distribución esté dirigida se incluye también otro tipo de software como procesadores de texto, hoja de cálculo, reproductores multimedia, herramientas administrativas, etcétera. En el caso de incluir herramientas del proyecto *GNU*, también se utiliza el término distribución *GNU/Linux*.

Existen distribuciones que están soportadas comercialmente, como *Fedora* (Red Hat), *openSUSE* (Novell), *Ubuntu* (Canonical Ltd.), *Mandriva*, y distribuciones mantenidas por la comunidad como *Debian* y *Gentoo*. Aunque hay otras distribuciones que no están relacionadas con ninguna empresa o comunidad, como es el caso de *Slackware* que ha sido creada por Patrick Volkerding.

1.2.2. El entorno de trabajo Shell

El trabajo por defecto con *Linux* se realiza introduciendo comandos de forma manual, *Linux* dispone de un programa especial *Shell* utilizado como una interfaz entre el usuario y el núcleo del Sistema Operativo (*Kernel*). A grandes rasgos, el *Kernel* es un programa que se carga en memoria en el momento en el que el PC se inicia, y se encarga de administrar el sistema hasta que apagamos el ordenador; es decir, crea y controla procesos, gestiona la memoria, el sistema de archivos y mucho más.

El *Shell* es un programa utilitario que se inicia cuando accedemos a nuestra sesión en el sistema operativo. Este, le permite al usuario interactuar con el *Kernel* a través de la interpretación de los comandos que el usuario ingresa en la línea de comandos ó a través de los "scripts" (archivos que ejecutan un conjunto de comandos).

Después de que tipeamos un comando, la responsabilidad del *Shell* es:

1. Analizar la línea de Comandos
2. Manejar los comodines, las redirecciones, tuberías y planificar el control de ejecución de los comandos.
3. Buscar el comando, si lo encuentra, ejecutar el comando, en caso contrario emitir un error.

1.2.3. El entorno de trabajo X Window

Casi todas las distribuciones más recientes de *Linux* incluyen el sistema *X Window*, el cual es el encargado de controlar y manejar la interfaz de usuario.

El sistema *X Window* se compone de dos partes principales: el servidor *X* y el programa para la gestión de las ventanas. El servidor *X* es el programa que se encarga realmente de dibujar en la pantalla. Por el contrario, el gestor de ventanas, como su nombre indica, es el encargado de crear las ventanas y gestionar su apariencia, por lo que la apariencia de las aplicaciones variará según se use uno u otro gestor de ventanas, entre los que destacan por su sencillez de uso los entornos *GNOME* y *KDE*.

En la distribución de *Linux* con la que vamos a trabajar (*Ubuntu*) se carga por defecto el entorno gráfico *GNOME*.

1.3. Entrar en sesión

1. Cuando encienda el ordenador del aula aparecerá en pantalla las dos opciones de elección de sistema operativo. Elija la opción de *Ubuntu* (es la primera que aparece).
2. Una vez cargado el sistema operativo, aparecerá una ventana en la que deberá elegir la opción **otros** que le pedirá su usuario y su contraseña. Como usuario introduzca su identificador de la forma:

`uoxxxxxx@uniovi.es`

y a continuación introduzca su contraseña (previsiblemente suministrada por el Vicerrectorado de Informática).

1.4. Linux en modo gráfico

En *Linux* todo lo guardado en un dispositivo de almacenamiento permanente se trata como un fichero (archivo), desde los ficheros de datos hasta las particiones de los discos pasando por el ratón y la tarjeta de sonido. Los directorios (carpetas) también son ficheros.

El sistema de archivo de *Linux* sigue todas las convenciones de *Unix*, lo cual significa que tiene una estructura determinada, compatible y homogénea con el resto de los sistemas *Unix*. Al contrario que en Windows o MS-DOS, el sistema de archivos en cualquier sistema *Unix* no está ligado de una forma directa con la estructura del hardware.

Todo el sistema de archivos de *Unix* tiene un origen único en un directorio denominado “raíz” o “root” y representado por `/`. Bajo este directorio se encuentran todos los ficheros a los que puede acceder el sistema operativo. Estos ficheros se organizan en distintos directorios cuya misión y nombre son estándar para todos los sistemas *Unix*.

Algunos de los directorios más interesantes son:

- `/` : Raíz.
- `/usr`: Aquí se encuentra la gran mayoría de los ficheros existentes en un sistema Linux, tales como documentación, casi la totalidad de los comandos de usuario, juegos, librerías, etc.
- `/bin`: Aquí están los comandos que pueden usar todos los usuarios (incluido el administrador — también llamado superusuario o *root*).
- `/sbin`: Aquí están los comandos que sólo puede usar el administrador o *root*.
- `/dev`: Ahí están todos los dispositivos de nuestra máquina.
- `/home`: Lugar donde se almacenan las cuentas de usuarios.
- `/lib`: Enlaces a las bibliotecas (también llamadas librerías) que se necesitan para el sistema.
- `/var`: Contiene información variable, como por ejemplo los informes (*logs*) del sistema (`/var/log`), correo local, etc.

- **/tmp**: Directorio temporal (que se borra cada vez que se reinicia el ordenador).
- **/etc**: Configuración global de los programas.
- **/root**: Cuenta del administrador.
- **/boot**: Aquí está todo lo necesario para arrancar el sistema.
- **/proc**: Sistema de ficheros virtual de información de procesos y del kernel.

En el menú situado en el lateral izquierdo del escritorio (en otros ordenadores puede estar en la parte inferior o superior, pues es configurable), conocido como *lanzador*, encontrará diferentes iconos que le permitirán acceder a diferentes aplicaciones y carpetas.

El icono inferior del *lanzador* representa el acceso al menú de inicio, desde donde podrá buscar cualquier aplicación. En el *lanzador* también encontrará un icono que representa una carpeta que le permite acceder directamente a su carpeta personal. Desde su carpeta puede acceder al sistema de archivos a través de la barra de herramientas que aparece en el lateral izquierdo. Una vez activado el sistema de archivos, explore los directorios anteriormente citados. Es posible que intente acceder a lugares del disco duro no visibles para usted debido a los permisos de acceso (que luego veremos). Pruebe también diversas aplicaciones que están instaladas en el sistema a partir del menú.

En su sistema (si usted es superusuario) puede instalar más aplicaciones mediante el [Centro de software de Ubuntu](#) cuyo icono de acceso se instala automáticamente en el lanzador. Posteriormente es conveniente instalar el [Gestor de paquetes Synaptic](#) y utilizar éste para instalar software.

1.5. Linux en modo terminal

Para acceder al modo Terminal debemos abrir un (o una) terminal. Para ello vamos al buscador del menú de inicio y escribiremos [Terminal](#) para poder acceder al enlace en el que Linux abre una ventana de terminal en modo texto.

Un atajo de teclado para realizar esta tarea es: <ALT+CTRL+t>.

Se pueden abrir más “solapas” dentro de la misma ventana mediante el menú:

[Archivo](#), [Abrir pestaña](#)

Un atajo de teclado para realizar esta tarea es: <MAYUS+CTRL+t>.

Cada solapa es independiente de las demás. En cada una de ellas está funcionando un programa que se denomina *intérprete de comandos* o *shell*. En nuestro caso éste nos indica que está esperando por nuestras órdenes con \$, el *prompt*.

También nos indica el usuario, el ordenador en que estamos (si tiene nombre) y el directorio de trabajo (todo esto es configurable). En nuestro caso deberá aparecer algo parecido a:

```
IDENT\uo ...@aula_nombre_aula.ubuntu_numero_de_ordernador.uniovi.es:~$
```

El directorio de trabajo (que luego describiremos con más detalle) cuando se abre la terminal es el directorio personal del usuario (que suele estar dentro de `/home`. El directorio personal se indica en la shell mediante el símbolo¹ `~` (que se consigue en el teclado pulsando a la vez `<ALTGR+4>`).

Para salir de una solapa (o del modo terminal si sólo tenemos una solapa abierta) se pulsa `<CTRL+D>` (la tecla Control a la vez que la tecla “d”) o el comando `exit`.

Los comandos del *shell* de Unix son de la forma:

```
$ orden parámetro1 parámetro2 ...
```

pulsando al final la tecla de `<Intro>` o `<Entrar>`.

Existen comandos que nos permiten saber en qué directorio estamos, cambiar de directorio, etc. Algunos de estos comandos son:

- **pwd**. Muestra el directorio actual. Su sintaxis es `pwd`.
- **ls**. Muestra todos los ficheros en la carpeta con colores para identificar el tipo (todo esto es configurable): verde para los ejecutables, azul las carpetas, fucsia las imágenes, rojo los comprimidos, etc.
- **cd**. Viene del inglés *change directory*, con el podemos cambiar el directorio actual. Para ir al directorio padre del actual usamos `cd ..`. Para ir a un directorio que cuelga del actual su sintaxis es `cd directorio`.
- **mkdir**. Este comando crea un directorio. Su sintaxis es `mkdir directorio`.
- **rmdir**. Sirve para borrar directorios. Si el directorio no está vacío da error. Su sintaxis es `rmdir directorio`.

Ejemplo 1.5.1. *Con el fin de familiarizarnos con el uso de estos comandos lo primero que debemos hacer es abrir el Terminal. En algunas versiones del sistema operativo puede aparecer en el lanzador un enlace que nos abre directamente una terminal, si no es así, realice las siguientes acciones:*

1. Active el menú `inicio`.
2. Escriba en el buscador `Terminal`. A partir de ahora las órdenes las escribirá en el Terminal que acaba de abrir.
3. Teclee en el Terminal `pwd` y pulse `<Intro>`.
Observe como en la primera línea aparece el comando tecleado, en la segunda línea muestra la respuesta del sistema indicándonos en el directorio en el que estamos y en la tercera línea podemos teclear un nuevo comando.
4. Teclee `ls` y pulse `<Intro>`.
Nos indica que en el directorio en el que nos encontramos hay varias carpetas. Básicamente la información obtenida con este comando es el contenido del directorio. Si nos interesa obtener

¹Denominado “tilde” en inglés.

más información como por ejemplo, permisos, propietarios, tamaño, fecha y hora de creación de cada fichero deberá utilizar el parámetro `-la`.

5. Teclee `ls -la` y pulse <Intro>.

6. Teclee `cd Escritorio` y pulse <Intro>.

Observe cómo ha cambiado de directorio y el control ahora se halla ubicado en el directorio Escritorio.

7. Teclee `cd ..` y pulse <Intro>.

Ha vuelto al directorio anterior.

8. Para cambiar al directorio raíz teclee `cd /` y pulse <Intro>.

9. Para comprobar que ahora está en el directorio raíz escriba `ls`.

10. Realice las operaciones necesarias para volver a su carpeta personal.

11. Una vez en su directorio personal escriba `mkdir midirectorio` y pulse <Intro>.

12. Para saber si se ha creado el directorio teclee `ls` y pulse <Intro>.

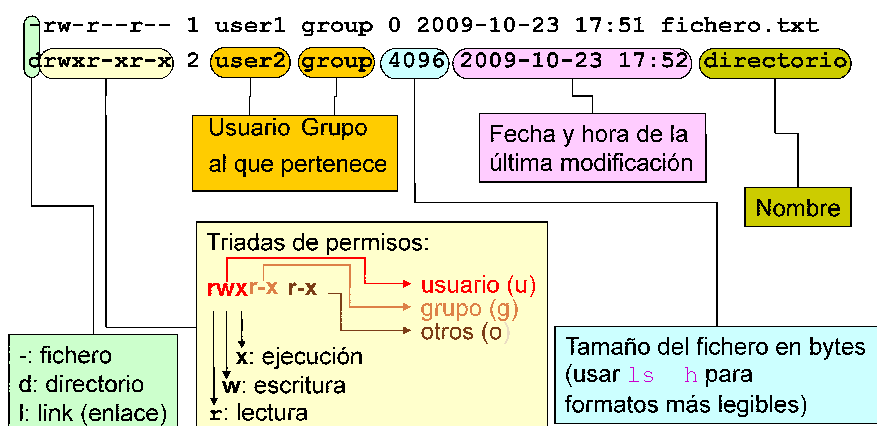
Observe cómo junto con los directorios anteriores aparece el que acaba de crear.

13. Escriba `rmdir midirectorio` y pulse <Intro>.

14. Para comprobar que se ha borrado el directorio teclee `ls` y pulse <Intro>.

Ficheros en Linux - Permisos

Al hacer un listado largo de un fichero (`ls -l`) o directorio (`ls -ld`) obtenemos:



1.6. Trabajo básico con ficheros en modo terminal

Existen comandos que le permitirán copiar el contenido, mover y eliminar ficheros. A continuación veremos el funcionamiento de los mas comunes.

- **cp**. Copia un fichero a otro directorio. Es bastante fácil de recordar y muy útil. Su sintaxis es `cp fichero destino`.
- **mv**. Mueve un fichero a otro lugar. Su sintaxis es `mv fichero destino`. Ojo: si el destino existe normalmente Unix no pregunta y lo sobrescribe (perdiéndose el contenido anterior).
- **rm**. Sirve para borrar ficheros y carpetas. Tiene algunas opciones interesantes como `-r` que borra recursivamente todo lo que hay en un directorio y `-f`, que borra sin pedir confirmación del usuario. Su sintaxis es `rm fichero` o `rm -fr directorio`. Por defecto, muchos sistemas Unix, no piden confirmación de borrado (y se pierde el fichero irremediablemente).
- **cat**. Muestra el contenido de un fichero. Su sintaxis es `cat fichero`.
Cuando el contenido del fichero es muy grande, con el comando `cat` perderíamos gran cantidad de información, ya que no podríamos leer el contenido completo del fichero. Para evitar ese problema existe el comando **more**.
- **more**. Su sintaxis es `more fichero`.
- **grep**. Permite localizar una cadena dentro de un fichero. Su sintaxis es `grep cadena fichero`. Si quiere que el comando encuentre la cadena independientemente de si se escribe en mayúsculas o minúsculas deberá añadir el parámetro `-i`.
- **du**. Muestra el tamaño ocupado en disco de un subdirectorio o fichero. Su sintaxis es `du -h fichero/subdirectorio`.
- **find**. Este comando permite buscar un fichero dentro de la estructura de directorios, su utilización es muy extensa y tiene gran variedad de argumentos. Si se quieren buscar todos los ficheros de una extensión determinada que se encuentren en todo el disco se utilizará la siguiente sintaxis: `find / -name "*.nombre_de_la_extensión"`.

Ejemplo 1.6.1. *Para familiarizarnos con el uso de estos comandos crearemos dos directorios y copiaremos un fichero de un directorio a otro.*

1. Abra el terminal.
2. Teclee `mkdir midirectorio` y pulse `<Intro>`.
3. Teclee `ls` y pulse `<Intro>`.
Vea en el listado el directorio que acaba de crear.
4. Teclee `cd midirectorio` y pulse `<Intro>`.
Ahora se encuentra en el directorio recientemente creado.

5. Teclee `gedit` y pulse `<Intro>`.
Observe que se abre el **editor de texto** que se instala por defecto con Ubuntu.
6. Escriba en el editor `hola`.
7. Guarde el fichero con el mismo nombre `hola.txt`. Hemos utilizado la extensión `txt` para indicar que se trata de un fichero de texto.
8. Salga del editor.
9. Teclee `mkdir practica` y pulse `<Intro>`.
10. Teclee `cp hola.txt practica` y pulse `<Intro>`.
11. Teclee `cd practica` y pulse `<Intro>`.
12. Teclee `ls` y pulse `<Intro>`.
Observe que ahora tiene un fichero `hola.txt` en `practica` y otro fichero `hola.txt` en `midirectorio`.
13. Teclee `cd ..` y pulse `<Intro>`.
Ha vuelto al directorio `midirectorio`.

Ejemplo 1.6.2. En este ejemplo moveremos un fichero de un directorio a otro.

1. Teclee `nano` y pulse `<Intro>`. El editor `nano` está normalmente incluido en las distribuciones Linux y es sencillo de utilizar en modo terminal.
2. Escriba en el editor la palabra `adios`.
3. Guarde el fichero con el mismo nombre (`adios.txt`).
4. Salga del editor.
5. Teclee `mv adios.txt practica` y pulse `<Intro>`.
6. Teclee `ls` y pulse `<Intro>`.
7. Teclee `cd practica` y pulse `<Intro>`.
8. Teclee `ls` y pulse `<Intro>`.
Observe que ahora el fichero `adios.txt` de `midirectorio` se ha movido al directorio `practica`.

Ejemplo 1.6.3. Borrando ficheros y directorios.

1. Teclee `cd ..` y pulse `<Intro>`.
2. Escriba `rm practica` y pulse `<Intro>`.
Aparecerá un mensaje de error indicándole que `rm` no puede borrar un directorio.

3. Teclee `rmdir practica` y pulse `<Intro>`.
Aparecerá un nuevo mensaje de error indicándole que no se puede borrar el directorio porque no está vacío.
4. Teclee `rm -r practica` y pulse `<Intro>`.
5. Teclee `ls` y pulse `<Intro>`.
Observe que el directorio `practica` (y todo lo que “colgaba” de él) ha sido borrado.
6. Escriba `rm hola.txt` y pulse `<Intro>`.
Observe que el fichero `hola.txt` ha sido borrado.

Ejemplo 1.6.4. Otras operaciones con ficheros.

1. Teclee `cat /usr/share/dict/words` y pulse `<Intro>`.
2. Teclee `more /usr/share/dict/words` y pulse `<Intro>`. *Observe la diferencia.*
3. Teclee `gedit` y pulse `<Intro>`.
4. Escriba en el editor las palabras `Hola mundo`.
5. Guarde el fichero con el nombre `hola.txt`.
6. Salga del editor.
7. Teclee `grep hola hola.txt` y pulse `<Intro>`.
Si no encuentra la cadena teclee:
8. Teclee `grep -i hola hola.txt` y pulse `<Intro>`.
9. Teclee `grep mundo hola.txt` y pulse `<Intro>`.
10. Teclee `du -h hola.txt` y pulse `<Intro>`.
11. Teclee `du -h midirectorio` y pulse `<Intro>`.
12. Escriba `find / -name "*.txt"`.

1.7. Otros comandos interesantes

- **date**. Muestra la fecha y hora del sistema.
- **cal**. Genera un calendario. Con la sintaxis `cal`, se genera el calendario del mes actual y si se indica el mes y el año por ejemplo, `cal 12 2011` facilita un calendario de diciembre de 2011.

- **man**. Proporciona un manual en línea. Muestra en pantalla la misma información que figura en los manuales. La sintaxis tiene dos formas posibles:

```
man nombre_comando
```

```
man -s número_de_manual nombre_comando
```

siendo la primera la más utilizada.

Los manuales estándar de UNIX van numerados del 1 al 7:

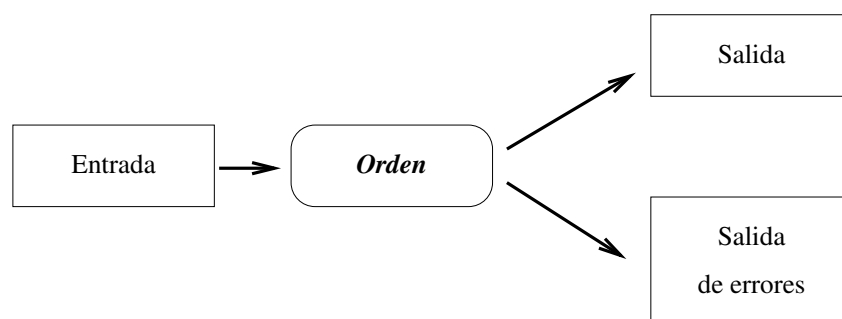
1. Comandos de uso general.
2. Llamadas al sistema.
3. Rutinas de las bibliotecas del sistema.
4. Formatos de los ficheros.
5. Miscelánea (cabeceras, macros, caracteres, etc.).
6. Juegos.
7. Ficheros especiales de dispositivos.
8. (No existe).
9. Glosario de términos empleados en los manuales.

Ejercicio 1.7.1. Utilice los diversos comandos que ya conoce: `cal`, `more`, `cat`, `grep`, etc.

Ejercicio 1.7.2. Obtenga con el manual en línea información de los comandos estudiados.

1.8. Ficheros estándar y redireccionamiento

La ejecución de un comando típico responde al siguiente esquema:



Un comando, si necesita algún dato de entrada, va a esperar que este sea introducido a través de teclado, que es la **entrada estándar** (o por defecto) en el sistema operativo, que se denomina `stdin`.

Si el comando ha de devolver alguna información respecto al resultado conseguido por su ejecución lo enviará al dispositivo de **estándar de salida**, que se denomina `stdout`, y por defecto es la pantalla.

De la misma manera, posibles errores en la ejecución del comando saldrán por el dispositivo de **estándar de error**, que se denomina `stderr` que también es la pantalla (por defecto).

Este comportamiento por defecto de los comandos (entrada por teclado, salida por pantalla) puede ser modificado a través de la denominada “redirección”.

Lo que se ha expuesto no quiere decir que un programa u orden concreta no manejen otros ficheros diferentes, tanto de entrada y salida. Por ejemplo, la orden `cat` permite concatenar varios ficheros. Si ponemos:

```
cat datos1.txt datos2.txt
```

la orden `cat` lee el contenido del fichero `datos1.txt` y lo muestra en el fichero estándar de salida, y posteriormente lee el contenido del fichero `datos2.txt` y lo muestra en el fichero estándar de salida a continuación del anterior.

1.8.1. Redirección simple

Puede variarse la entrada/salida estándar, redirigiéndola de/a un fichero, utilizando el formato:

- `comando < entrada`
`entrada`: fichero que sustituye a `stdin`
- `comando > salida`
`salida`: fichero que sustituye a `stdout`
- `comando >> salida`
La utilización de `>>` en lugar de `>` significa añadir al final del fichero, si ya existe, en vez de al principio.
- `comando 2> errores`
`errores`: fichero que sustituye a `stderr`
- `comando 2>> errores`

(y las diversas combinaciones).

Ejemplo 1.8.1. *Familiarizarse con el uso del redireccionamiento de entrada y salida.*

1. Teclee en la terminal `ls > z`.

Se genera un listado del directorio y lo pone en el fichero "z". Si "z" ya existía la información anterior se pierde.

2. Teclee `cat z`.

Nos muestra el contenido del archivo "z".

3. Teclee `ls -l >> z`.

Se genera un listado más detallado del directorio y lo añade en el fichero "z".

4. Teclee `ls abc* 2> z1`.

Se genera un listado del directorio de los ficheros que empiezan por "abc" (ya veremos el significado de ""), lo muestra en pantalla y si hay error el mensaje correspondiente lo pone en el fichero "z1". (Pruébese con un directorio en el que no exista ningún fichero que empiece por "abc". Mírese después "z1" con el comando `cat`.)*

1.8.2. Redirección encadenada

Permite que la salida de un comando se utilice como entrada al siguiente. El formato es el siguiente:

`comando1 | comando2 | ... | comando_N`

que se denomina *pipeline* o "tubería". La salida estándar de un comando de la "tubería" actúa como entrada estándar del siguiente. No tiene sentido redireccionar la entrada, salvo en el primer comando. No tiene sentido redireccionar la salida, salvo en el último comando.

Ejemplo 1.8.2. *Utilización de redirección encadenada.*

1. Teclee `cat /usr/share/dict/words | more`.

La salida de `cat` es la entrada para `more`, por eso el comando `more` no necesita el nombre de fichero.

2. Teclee `ls -l | grep z`.

`grep z` buscará la cadena solicitada en la salida de `ls -l`.

1.9. Referencias absolutas y relativas en el árbol de directorios

Como ya hemos visto anteriormente, el contenido de un disco duro (u otra unidad de almacenamiento típica) se referencia en forma de un árbol invertido con la raíz en la parte de arriba. Las diversas ramas del árbol son los directorios. Las hojas son ficheros concretos que "cuelgan" de las ramas.

Puede haber tantas ramas y hojas como se desee (hay un límite pero es muy elevado como para que nos preocupemos), o sea puede haber tantos directorios y fichero normales como deseemos.

Para referirnos a un fichero concreto deberemos indicar el **camino de búsqueda absoluto** o completo. Por ejemplo:

```
/home/marcos/electromagnetismo/tema1/problemas/coulomb.txt
```

se refiere al fichero de nombre **coulomb.txt**, dentro del directorio **problemas**, que a su vez está dentro del directorio **tema1**, y así sucesivamente, hasta el directorio **home** que “cuelga” directamente del directorio raíz **/**.

Como es largo y difícil teclear todo lo anterior para acceder a los datos contenidos en **coulomb.txt** se ha inventado el concepto de **directorio de trabajo**, al que se accede como ya hemos visto con la orden **cd**. El directorio de trabajo le dice al *shell* que cuando le pasamos un nombre de fichero que **NO empieza** por **/** el nombre es un **camino de búsqueda relativo** y debe añadir al mismo el camino de búsqueda que lleva desde el directorio raíz **/** hasta el directorio de trabajo.

Por ejemplo, si el directorio de trabajo es:

```
/home/marcos/electromagnetismo/tema1/problemas
```

entonces **cat coulomb.txt** nos mostrará en pantalla el fichero:

```
/home/marcos/electromagnetismo/tema1/problemas/coulomb.txt
```

Si el directorio de trabajo es:

```
/home/marcos/electromagnetismo/tema1
```

entonces **cat problemas/coulomb.txt**

nos mostrará el mismo fichero. Sin embargo, si en este caso tecleamos:

```
cat coulomb.txt
```

nos intentará mostrar:

```
/home/marcos/electromagnetismo/tema1/coulomb.txt
```

que es posible que no exista (pero que si existe evidentemente no es el fichero que nos interesa).

Para complicar aun más las cosas, todos los directorios contienen dos directorios especiales:

- El directorio actual, representado por un punto (**.**).
- El directorio padre, representado por dos puntos (**..**).

Supongamos que del directorio **tema1** cuelgan los directorios **problemas** y **teoria**. Para acceder al fichero **coulomb.txt** que se encuentra en el directorio **problemas** desde el propio directorio utilizaríamos:

```
cat coulomb.txt
```

o bien:

```
cat ../coulomb.txt
```

Para acceder al fichero `coulomb.txt` que se encuentra en el directorio `problemas` desde el directorio `teoria` utilizaríamos:

```
cat /home/marcos/electromagnetismo/tema1/problemas/coulomb.txt
```

o bien:

```
cat ../problemas/coulomb.txt
```

Ejercicio 1.9.1. Cree en el directorio `Escritorio` un directorio que se llame `padre`. Cree un nuevo directorio que se llame `hijo` que cuelgue del anterior. Edite un fichero con `gedit` que contenga la frase *Estoy aprendiendo a hacer referencias a ficheros*, guarde el fichero con el nombre de `ejercicio.txt` en el directorio `hijo`.

1. Posicionándose en el directorio `padre` y utilizando la referencia relativa, mueva el fichero `ejercicio.txt` del directorio `hijo` a `padre`. Compruebe que el fichero se ha movido.
2. Posicionándose en el directorio `hijo` y utilizando la referencia relativa, mueva el fichero `ejercicio.txt` del directorio `padre` a `hijo`.

1.10. Caracteres comodín en los nombres de ficheros

El *shell* tiene un manejo de ficheros bastante potente. Por ejemplo, puede manejar múltiples ficheros de una vez. Para ello usa lo que se denominan **caracteres comodín**.

No vamos a exponer este tema completamente. Sólo indicaremos que el carácter “*” en un nombre de fichero indica cualquier cadena de caracteres (incluso la cadena vacía), por lo que una orden como:

```
ls a*b*c
```

indica que debe listar el contenido del directorio pero solamente los ficheros que empiecen por “a”, tengan una “b” en alguna parte por el medio y acaben por “c”.

Otro carácter comodín es “?” que indica un carácter cualquiera (pero uno solo). Por ejemplo,

```
ls ?xy*z
```

indica que debe listar el contenido del directorio pero solamente los ficheros que empiecen por un carácter cualquiera, luego tengan “xy” en el nombre, a continuación cualquier cadena de caracteres y acaben por “z”.

Ejercicio 1.10.1. Compruebe en la terminal el uso de los caracteres comodín.

1.11. Propietarios de los ficheros y sus permisos de acceso

Linux es un sistema operativo multitarea y multiusuario. Esto quiere decir que es capaz de ejecutar varios programas (o tareas) de forma simultánea y albergar a varios usuarios de forma simultánea. Por

lo tanto, todos los usuarios de *Linux* deben tener una cuenta de usuario en el sistema que establezca los privilegios del mismo. A su vez *Linux* organiza a los usuarios en grupos de forma que se puedan establecer privilegios a un determinado grupo de trabajo, para el acceso a determinados archivos o servicios del sistema. Por lo tanto, todo fichero en Unix tiene un propietario, que pertenece a su vez a un grupo (o más de un grupo) de usuarios. Para cada usuario existen por tanto tres tipos de usuarios:

1. Él mismo (**u**).
2. Los de sus grupos (**g**).
3. El resto, otros (**o**).

De esta manera se puede permitir a ciertos usuarios ver la información contenida en nuestro fichero si lo deseamos, y ocultarla a otros. Los permisos posibles son de **lectura** (**r**), **escritura** (**w**), que incluye permiso de borrado, y de **ejecución** (**x**), que se aplica a programas y también a directorios (aunque en este último caso no significa lo mismo que para los programas),

Las órdenes que manejan permisos son las siguientes:

- **chown** cambia el propietario de un fichero (deberá ser nuestro para poderlo hacer). La sintaxis es la siguiente:

```
chown nuevo_propietario fichero(s)
```

- **chgrp** cambia el grupo de un fichero. La sintaxis es la siguiente:

```
chgrp nuevo_grupo fichero(s)
```

- **chmod** cambia los permisos de acceso de un fichero. Actúa sobre el propietario (**u**), el grupo (**g**), o el resto (**o**), añadiendo (+) o quitando (−) los permisos (**rw**x). Por ejemplo:

```
chmod o+r fichero
```

 Se da permiso de lectura al resto de usuarios.

```
chmod g+wx fichero
```

 Se da permiso de escritura y ejecución al grupo.

```
chmod o-rwx fichero
```

 Se quitan todos los permisos al resto de usuarios.

Ejercicio 1.11.1. Compruebe los permisos del fichero `ejercicio.txt` anteriormente creado, proporcione permiso de escritura al grupo y al resto de usuarios.

Ejercicio 1.11.2. Compruebe los permisos del directorio `hijo`, quite todos los permisos al resto de usuarios.

1.12. Ejecución de procesos en Unix

1.12.1. Lanzado de programas en modo normal

Para ejecutar un programa en modo Terminal se teclea su nombre después del *prompt*, por ejemplo `/usr/bin/cal` y se pulsa <Intro> para que la Shell ejecute la orden. En este ejemplo le decimos a la Shell dónde está el fichero que queremos ejecutar.

Cuando le decimos a la Shell exactamente dónde está el fichero ejecutable va a buscarlo ahí mismo y lo ejecuta (si es ejecutable y tiene el permiso de ejecución activo). Por ejemplo

```
./sumar
```

busca el fichero `sumar` en el directorio de trabajo y lo ejecuta, mientras que

```
/home/marcos/restar
```

busca el fichero `restar` en el directorio `/home/marcos` y lo ejecuta (si puede).

Si tecleamos una orden sin referencia directa al fichero ejecutable la Shell debe buscar la orden en algún lugar (podría ser una orden interna, almacenada en memoria, o un programa ejecutable en el disco—como es el caso de `ls`).

Si no está en memoria (Unix no suele almacenar en memoria órdenes—como si hace MS-DOS, por ejemplo), debe buscarla en el disco. En Unix existe una variable de Shell, denominada `PATH` (“camino” en inglés), que guarda información de dónde debe ir a buscar las órdenes. Consiste en una cadena de caracteres en los que se indican directorios separados por el carácter `:`. Por ejemplo:

```
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games
```

indica que al ejecutar, por ejemplo, `$ ls <Intro>`, la Shell empieza a buscar en:

```
/usr/local/sbin  
/usr/local/bin  
/usr/sbin  
/usr/bin  
/sbin  
/bin  
/usr/games
```

en ese orden, y en cuanto encuentra un fichero ejecutable denominado `ls` en alguno de esos directorios deja de buscar y lo ejecuta. Concretamente `ls` está en `/bin`. En ese momento la Shell lanza un nuevo proceso que ejecuta `ls`. Si la orden dada no está en el “`PATH`” nos muestra un mensaje de error.

¿ Pero cómo podemos saber dónde está un fichero ejecutable concreto dentro de esa lista de directorios? El comando usado es `whereis`. En nuestro caso `$ whereis ls<Intro>` nos muestra:

```
ls: /bin/ls /usr/share/man/man1/ls.1.gz
```

o sea el directorio donde está el ejecutable y también la ayuda del manual si es que existe.

1.12.2. Lanzado de programas en modo desatendido

En el modo normal de lanzar programas si se cierra la terminal (de algún modo) sus programas “hijos”² acaban intempestivamente.

²Que son los programas que se lanzan desde ella.

Si queremos cerrar la terminal y salir de sesión hasta otro momento y queremos mantener programas en ejecución (por ejemplo, un programa que dura días) debemos lanzarlo de una manera especial.

Para ello se utiliza el comando `nohup`, de la manera siguiente:

```
$ nohup programa
```

Entonces el fichero estándar de salida pasa a ser `nohup.out` (en vez de la pantalla), funcionando en modo añadir.

1.12.3. Control de ejecución de los procesos

Todos los procesos en marcha en Unix tienen asignado un número (PID) y se puede obtener información de los mismos con la orden `ps` (*process status*). Asimismo se puede obtener una tabla con información con el comando `top` (*table of processes*).

La orden `ps`, sin opciones solamente dará información sobre los procesos puestos en marcha desde mi terminal actual. Existen multitud de opciones, algunas de las más comunes son:

`ps -e`: Información de todo el sistema

`ps -l`: Formato largo

`ps -aux`: Da información completa

Según las opciones que le suministremos al comando, aparecerán diversos campos, entre los que cabe destacar:

UID: Número de identificación del usuario

PID: Número de identificación del proceso

PPID: Número de identificación del proceso padre (aquel proceso que creó a éste)

PRI: Prioridad (cuanto más alto sea este número, menor es la prioridad)

NI: Número “nice” (es la prioridad efectiva; sólo se puede subir o lo que es lo mismo, bajar la prioridad)

TIME: Tiempo de CPU usado hasta ese momento en `min:seg`.

COMD: Nombre del comando que se está ejecutando (como se “lanzó”).

Ejercicio 1.12.1. Compruebe en la terminal el uso de los comandos anteriormente expuestos `ps`, `ps -e`, `ps -l`, `ps -aux` y `top`.

Ejercicio 1.12.2. Pruebe el comando: `ps -aux | grep nombre_de_usuario` sustituyendo el nombre de usuario por su propio nombre de usuario. ¿Qué muestra ese comando?

Si se quiere detener definitivamente un proceso se puede utilizar el comando `kill`. Su sintaxis es:

```
kill número_de_proceso
```

Ejercicio 1.12.3. Ejecute en una terminal `gedit` y utilice el comando `kill` para parar el proceso correspondiente al `gedit`.

Ejercicio 1.12.4. Cree tres directorios llamados `IFC`, `Teoria` y `Practica`. El directorio `IFC` debe colgar de su directorio de usuario. Los dos últimos deben de colgar del directorio `IFC`.

1. Con el redireccionamiento simple de salida utilice el comando `ls` para crear un fichero denominado `listado.txt` que contenga todos los archivos de su directorio de usuario.
2. Desde el directorio `IFC` mueva el fichero `listado.txt` al directorio `Practica` utilizando las referencias relativas al árbol de directorios.
3. Compruebe los permisos del archivo `listado.txt`. Cambie los permisos de dicho fichero de manera que todos tengan permiso de lectura y escritura.
4. Si no tiene el editor `gedit` abierto, vuelva a abrirlo y sin cerrarlo compruebe los procesos y obtenga el número de proceso para finalizarlo mediante la orden `kill`.
5. Borre todos los directorios y ficheros creados a lo largo de toda la práctica.